

Pix Race do zero com o Reversa

Guia passo a passo. Linha verde em fonte de máquina é exatamente o que você digita. Linha em itálico é o que o Reversa pergunta. Modo guiado: cada agente termina pedindo CONTINUAR.

Parte 0. Preparar a pasta (terminal)

1. Criar o projeto e colocar os dados

Os dados não são gerados. A pasta data/ está no repositório github.com/sandeco/pix-golpe (o transacoes.bin via Git LFS) e o .env com as chaves você recebe à parte.

Você	<code>mkdir pix-race</code>
Você	<code>cd pix-race</code>
Reversa	<i>Copie para dentro de pix-race a pasta data/ (mensagens.jsonl, transacoes.bin, chaves.json, meta.json) e o arquivo .env com TYPESAFE_API_KEY e DEEPSEEK_API_KEY.</i>
Reversa	<i>Não crie nada dentro de data/. Os arquivos são usados como estão.</i>

2. Instalar o Reversa no projeto

Instala todos os agentes em .claude/skills/. Responda nome e idioma quando o instalador perguntar.

Você	<code>npx reversa install</code>
Reversa	<i>Qual seu nome? Qual idioma?</i>
Você	<code>Seu nome, depois pt-BR</code>

3. Abrir o Claude Code na pasta

Tudo daqui em diante é digitado dentro do chat do Claude Code.

Você	<code>c laude</code>
------	----------------------

Parte 1. Da ideia às specs (/reversa-new)

4. Abrir o pipeline greenfield

O Reversa pergunta o modo e pede o brief. Escolha Guiado para ver cada etapa.

Você	<code>/reversa-new</code>
Reversa	<i>Como você quer executar? 1 Guiado, 2 Expresso, 3 Outro</i>
Você	<code>1</code>
Reversa	<i>O que você quer construir?</i>

Você

Quero construir uma demonstração em tela dividida para mostrar, ao vivo, uma IA detectando golpe do Pix e rastreando o caminho do dinheiro até a quadrilha. Do lado esquerdo roda Rust com o Jev, da TypeSafe AI. Do lado direito roda Python com o DeepSeek. As mesmas mensagens em português entram nos dois lados ao mesmo tempo, cada lado decide se é golpe e, quando é, segue as transações num grafo dirigido de transações: cada conta é um nó, cada transação Pix é uma aresta com valor e instante, e o grafo fica em memória como listas de adjacência de saída em formato CSR (Compressed Sparse Row), ordenadas por tempo dentro de cada conta. São 10 milhões de transações e 1 milhão de contas. O espectador vê a corrida acontecer e compara velocidade, custo e acertos de cada abordagem.

Os dados já estão prontos na pasta data/, em quatro arquivos, e o projeto só lê. mensagens.jsonl tem uma mensagem por linha com os campos id, ts (segundos desde o início da simulação), remetente, texto, chave (a chave Pix citada) e golpe_real (verdade de referência, só para o placar, nunca enviada aos modelos). chaves.json é um mapa de chave Pix para id de conta. transacoes.bin é binário little-endian: cabeçalho PIX1, u32 n_contas, u32 n_transacoes, depois registros de 16 bytes com u32 origem, u32 destino, u32 valor_centavos e u32 ts, ordenados por ts. meta.json traz a verdade de referência de cada quadrilha: chave, nó, lista de laranjas, contas de saque e instante de início. Não é para criar gerador de dados nem alterar o que está em data/. Também vou passar um arquivo .env na raiz do projeto com as chaves TYPESAFE_API_KEY e DEEPSEEK_API_KEY. O Rust deve carregar esse .env com dotenvy e o Python com python-dotenv. Nunca gravar as chaves no código.

Cada lado da tela mostra a mensagem atual, o veredito (GOLPE, SUSPEITA ou OK), o placar de mensagens processadas e um grafo que acende quando o dinheiro é rastreado. Os golpes detectados vão para uma lista. Ao clicar num golpe dessa lista, abre uma tela de detalhe com a mensagem completa, o remetente, a probabilidade de golpe, o tipo de golpe, o modelo usado, os tokens gastos, o custo daquela decisão, e o rastreio no grafo dirigido: uma busca temporal em largura, com janela de 24 horas a partir da chave Pix, seguida de propagação de rótulos de suspeita, mostra de qual chave o dinheiro saiu, por quais contas laranja passou e em quais contas foi sacado, comparado com a verdade de referência. Quando a corrida termina ou é parada, um botão abre o relatório de acurácia: para cada lado, a matriz de confusão (real versus veredito), acurácia, precisão, recall e custo em dólares, mais uma linha com o gasto total da corrida somando os dois lados e uma conclusão em texto dizendo qual lado foi melhor.

5. Ideator (brainstorm em 6 perguntas)

Roda sozinho depois do brief. Gera _reversa_sdd/ideation.md.

Reversa *Qual problema isso resolve?*

Você **Ninguém vê na prática se IA tipada em Rust bate Python + LLM em detectar golpe do Pix e seguir o dinheiro.**

Reversa *O que o usuário consegue fazer que não conseguia?*

Você **Ver lado a lado, em tempo real, velocidade, custo e acerto de cada abordagem.**

Reversa *Quais alternativas existem?*

Você **Benchmarks em tabela. Não bastam porque ninguém vê a corrida acontecer.**

Reversa *Quem é o usuário?*

Você **Dev e analista antifraude que quer comparar stacks de IA.**

Reversa *Métrica de sucesso em 3 meses?*

Você **1000 mensagens processadas, rastreo idêntico nos dois lados, lado Rust termina em menos de 60 s.**

Reversa *Premissas perigosas?*

Você **Jev devolve JSON tipado estável. As 10 milhões de transações e 1 milhão de contas cabem em memória. DeepSeek responde em menos de 2 s.**

Reversa *Digite CONTINUAR para o Researcher.*

Você **CONTINUAR**

6. Researcher (personas)

Gera _reversa_sdd/personas.md. Uma persona basta.

Reversa *Quantas personas? 1, 2 ou 3*

Você **1**

Reversa *Qual o contexto dela no dia a dia?*

Você **Dev que grava demos e precisa mostrar a diferença entre as stacks num palco, sem enrolação.**

Reversa *Qual o nível técnico?*

Você **Avançado em Rust e Python, intermediário em LLMs.**

Reversa *Qual o objetivo final?*

Você **Convencer que tipagem + Rust vale a pena em antifraude.**

Reversa *Essa jornada faz sentido?*

Você **Sim, faz sentido.**

Reversa *Digite CONTINUAR para o Drafter.*

Você **CONTINUAR**

7. Drafter (PRD)

Gera `_reversa_sdd/prd.md`. Faz no máximo 2 perguntas de cobertura.

Reversa *Tem restrição de stack ou infraestrutura?*

Você **Servidor em Rust com axum e WebSocket. Worker em Python puro, sem numpy. Front-end em um único index.html.**

Reversa *Algo explícito fora do escopo?*

Você **Não gerar dados: ler os arquivos prontos em data/. Nada de banco de dados, nada de deploy, nada de dado bancário real.**

Reversa *Digite CONTINUAR para o Spec-SDD.*

Você **CONTINUAR**

8. Spec-SDD (specs com score)

Propõe a decomposição em componentes e escreve `_reversa_sdd/sdd/*.md` com nota de 0 a 100.

Reversa *Concorda com essa decomposição? Quer adicionar, remover ou renomear?*

Você **Concordo. Só garanta estes seis: leitura-dos-dados, decisao-por-ia, rastreo-no-grafo, corrida-em-tempo-real, interface-tela-dividida, relatorio-de-acuracia. Nenhum componente de geração de dados.**

Reversa *Specs geradas. Digite CONTINUAR para iniciar /reversa-forward.*

Você **CONTINUAR**

Parte 2. Das specs ao código (/reversa-forward)

O forward constrói uma feature por vez. Faça a primeira inteira, depois repita o passo 9 para cada feature seguinte. Se uma sessão fechar, digite /reversa-forward de novo: ele retoma de onde parou.

9. Abrir o ciclo forward e escolher a feature

Na primeira vez pergunta como organizar as specs. Depois pede a feature.

Você **/reversa-forward**

Reversa *Como você quer organizar as specs? 1 a 6*

Você **5**

Reversa *Descreva em uma frase a feature que você quer construir.*

Você **Leitura dos dados: carregar data/mensagens.jsonl, data/transacoes.bin, data/chaves.json e data/meta.json em memória no Rust e no Python, respeitando o formato descrito no brief, sem gerar nada.**

Reversa *Digite CONTINUAR para rodar /reversa-requirements.*

Você **CONTINUAR**

10. Requirements

Escreve requirements.md ancorado no PRD e nas specs. Dúvidas viram [DÚVIDA].

Reversa *requirements.md gerado com N dúvidas. Digite CONTINUAR.*

Você **CONTINUAR**

11. Clarify (só aparece se houver [DÚVIDA])	
Até 5 perguntas dirigidas. Responda curto.	
Reversa	<i>Exemplo: qual o formato exato do transacoes.bin?</i>
Você	Little-endian. Cabeçalho PIX1, u32 n_contas, u32 n_transacoes. Depois registros de 16 bytes: u32 origem, u32 destino, u32 valor_centavos, u32 ts, ordenados por ts.
Reversa	<i>Dúvidas resolvidas. Digite CONTINUAR para o plano.</i>
Você	CONTINUAR
12. Plan	
Gera roadmap.md, investigation.md e interfaces da feature.	
Reversa	<i>roadmap.md pronto. Digite CONTINUAR para decompor em ações.</i>
Você	CONTINUAR
13. To-do	
Quebra o roadmap em ações atômicas T001, T002... em actions.md.	
Reversa	<i>actions.md com M ações. Digite CONTINUAR para codar.</i>
Você	CONTINUAR
14. Coding	
Escreve o código, marca cada ação [X]. Aqui você mostra o cargo run na tela.	
Reversa	<i>M de M ações concluídas. Digite CONTINUAR para o sync.</i>
Você	CONTINUAR
15. Sync (fecha a feature)	
Registra o que foi entregue em _reversa_sdd/addenda/. Depois volte ao passo 9 com a próxima feature.	
Reversa	<i>Feature convergida. Para nova feature, use /reversa-forward.</i>
Você	/reversa-forward
Reversa	<i>Descreva em uma frase a feature que você quer construir.</i>
Você	Decisão por IA: cada mensagem passa pelo Jev no lado Rust e pelo DeepSeek no lado Python, respondendo 4 perguntas tipadas (golpe? tipo? urgência? pede Pix?), com latência medida.

Ordem sugerida das features seguintes, sempre pelo passo 9: rastreo no grafo, corrida em tempo real (servidor HTTP + WebSocket), interface em tela dividida, relatório de acurácia.

Parte 3. Documentação visual (/reversa-docs)

16. Mini-site do projeto	
Três perguntas, quatro agentes. Gera _reversa_docs/ com arquitetura 3D, glossário e página por feature.	
Você	/reversa-docs
Reversa	Pra quem é esse mini-site? 1 a 4
Você	1
Reversa	Qual profundidade? 1 a 4
Você	2
Reversa	Qual estilo visual? 1 a 5
Você	2
Reversa	Digite CONTINUAR (4 vezes, um por agente).
Você	CONTINUAR

Atalho: tudo de uma vez (modo expresso)

17. Da ideia ao código sem paradas	
Uma entrevista no início, depois roda todos os agentes até o código. Bom para fechar o vídeo.	
Você	/reversa-new expresso Demo em tela dividida: IA detecta golpe do Pix e rastreia a quadrilha. Esquerda Rust + Jev, direita Python + DeepSeek. Dados prontos em data/ e .env com as chaves. Não gerar dados.
Reversa	Entrevista única: responda os blocos (ideação, personas, restrições).
Você	As mesmas respostas dos passos 5 a 7
Reversa	Se surgirem dúvidas no caminho? 1 Não parar, 2 Parar e perguntar
Você	1
Reversa	Digite INICIAR para começar.
Você	INICIAR

Comandos de apoio: /reversa-agents-help explica cada agente. /reversa-add faz um ajuste pequeno depois do coding. /reversa-debugger registra um bug. npx reversa status mostra o estado no terminal.